



FOR JAVA DEVELOPERS MOVING TO NESTJS

Same group. Twice.

Spring Boot to NestJS, episode 27: Kafka with NestJS microservices

Moving a Kafka consumer is not moving a listener. The group, the failure policy and the wire contract are where the defaults stop matching.

Spring and Nest both configured with group payments handled ten messages twenty times: Nest appends -server to the group id.

With the suffix removed, Spring could not join the group at all: the two clients name their partition assignors differently.

A poison message: Spring retried ten times and moved on; Nest kept redelivering and blocked the partition. And Spring refused Nest's JSON until given a default type.

In scope

- @KafkaListener and @EventPattern
- Consumer groups across stacks
- Poison messages
- JSON on the wire

Not in scope

- Exactly-once and transactions
- Schema registries

At the time of recording Nest's Kafka transport uses kafkajs 2.2.4.

01 The measurements

BOTH GROUP payments, 10 MESSAGES	20 handled: groups payments and payments-server
postId "	Spring: InconsistentGroupProtocolException, never joined
POISON, THEN 5 GOOD, SAME KEY	Spring 10 attempts, 5 of 5; Nest 6 attempts, 0 of 5
POISON, GOOD ON OTHER KEYS	Nest: other partitions flowed
SPRING READS NEST'S JSON	refused without a default type; reads it with one

3 partitions, real processes.

02 Check the effective group

Nest appends a role suffix to the group id. Removing it matched the names, but a Spring and a Nest consumer still could not share the group in this setup: move whole groups.

```
main.ts

transport: Transport.KAFKA,
options: {
  client: { clientId: 'payments', brokers: [broker] },
  consumer: { groupId: 'payments' },
  postfixId: '',
},
```

both configured with group payments: 20 handlings for 10 messages

Run on a real machine, 2026-10-01, by scripts/verify-kafka.sh in the course repository.

03 Make the schema the contract

Spring's JSON serializer adds a `__TypeId__` header with the Java class name. For a topic two languages share, read by default type and ignore type headers.

```
application.properties

spring.kafka.consumer.value-
deserializer=org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
spring.kafka.consumer.properties.spring.deserializer.value.delegate.class=org.springframework.k
afka.support.serializer.JacksonJsonDeserializer
spring.kafka.consumer.properties.spring.json.use.type.headers=false
spring.kafka.consumer.properties.spring.json.value.default.type=com.example.Payment
```

with type headers ignored and a default type, Spring read both Spring's and Nest's messages

Run on a real machine, 2026-10-01, by scripts/verify-kafka.sh in the course repository.

Before you move a Kafka consumer

1. What group id does the broker actually see?
2. Can the old and new consumers share it? Test it
3. What happens to a message that always fails?
4. Does a stuck partition block others?
5. Is the payload contract a schema, or a Java class name?
6. Long handlers: check heartbeats

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 x https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-kafka.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	NestJS Kafka https://docs.nestjs.com/microservices/kafka

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. Apache Kafka is a trademark of the Apache Software Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-01 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.